

SIH26060

POLAR COMMAND

A build-ready Digital Twin platform for efficient remote management of India's Antarctic research stations - Maitri and Bharati.

Core Product Loop

OBSERVE -> UNDERSTAND -> PREDICT -> SIMULATE -> RESPOND

The product is not a 3D dashboard. The 3D twin is the spatial interface; the real value is the connected operational model linking environment, energy, infrastructure and logistics.

2 Stations

Maitri + Bharati

4 Domains

Environment,
Energy,
Infrastructure,
Logistics

1 Risk Engine

Cross-domain alerts
and dependencies

1 Simulator

What-if operational
scenarios

1. Executive Brief

Design objective: give a remote NCPOR operations team one operational picture of both stations and make the platform answer not only "what is happening?" but also "what is likely to happen?" and "what should we test before acting?"

Problem statement

SIH26060 asks for a Digital Twin framework for Maitri and Bharati integrating **infrastructure, energy, logistics and environmental monitoring** for efficient remote management. The public problem-statement listing identifies the Ministry of Earth Sciences (MoES) and the National Centre for Polar and Ocean Research (NCPOR) under the software category. [1]

MVP outcome

Build a web application where a user can switch between Maitri and Bharati, inspect a station twin, see live or simulated telemetry, understand cross-domain risks, forecast resource depletion, run what-if scenarios, manage logistics requests, and export an operational report.

Primary value

Operational decision support for remote management under constrained connectivity and limited on-site intervention.

Prototype strategy

Use public environmental data where available and clearly labeled synthetic telemetry for non-public operational domains. Keep live-data adapters replaceable.

Success in one sentence

Changing one condition should propagate through the twin. Example: colder weather -> higher heating demand -> higher power draw -> higher fuel burn -> shorter reserve runway -> higher logistics risk.

What the MVP is not

Not a weather app

Weather is an input to operational decisions.

Not an inventory CRUD

Logistics is tied to consumption, resupply windows and risk.

Not a 3D demo

The twin must connect spatial state to telemetry and simulation.

2. Product Principles

Principle	Implementation rule	Why it matters
Operational first	Every screen must answer a real station-management question.	Prevents a visually strong but operationally shallow demo.
Connected domains	Environment, energy, infrastructure and logistics share a common station state.	Creates a real Digital Twin story.
Explainable intelligence	Every alert shows contributing factors and thresholds/rules.	Operators need to trust why an alert exists.
Edge-ready	Local buffering + last-known-good state + delayed synchronization.	Remote connectivity cannot be treated as guaranteed.
Simulation before action	Operators can test generator, weather and resupply scenarios.	Moves product beyond passive monitoring.
Honest data	Mark demo telemetry as simulated; separate public observations from synthetic operational streams.	Avoids unsupported claims about confidential station systems.

Target users

User	Primary jobs	MVP access
NCPOR Operations	Monitor both stations, triage alerts, run scenarios, review reports.	Full operations
Station Engineer	Inspect energy and infrastructure health, acknowledge/resolve issues.	Technical views
Logistics Officer	Review stock, submit/approve requisitions, track deliveries.	Logistics + reports
Leadership	See station health, risk summary and executive reports.	Read-only overview

Primary user stories

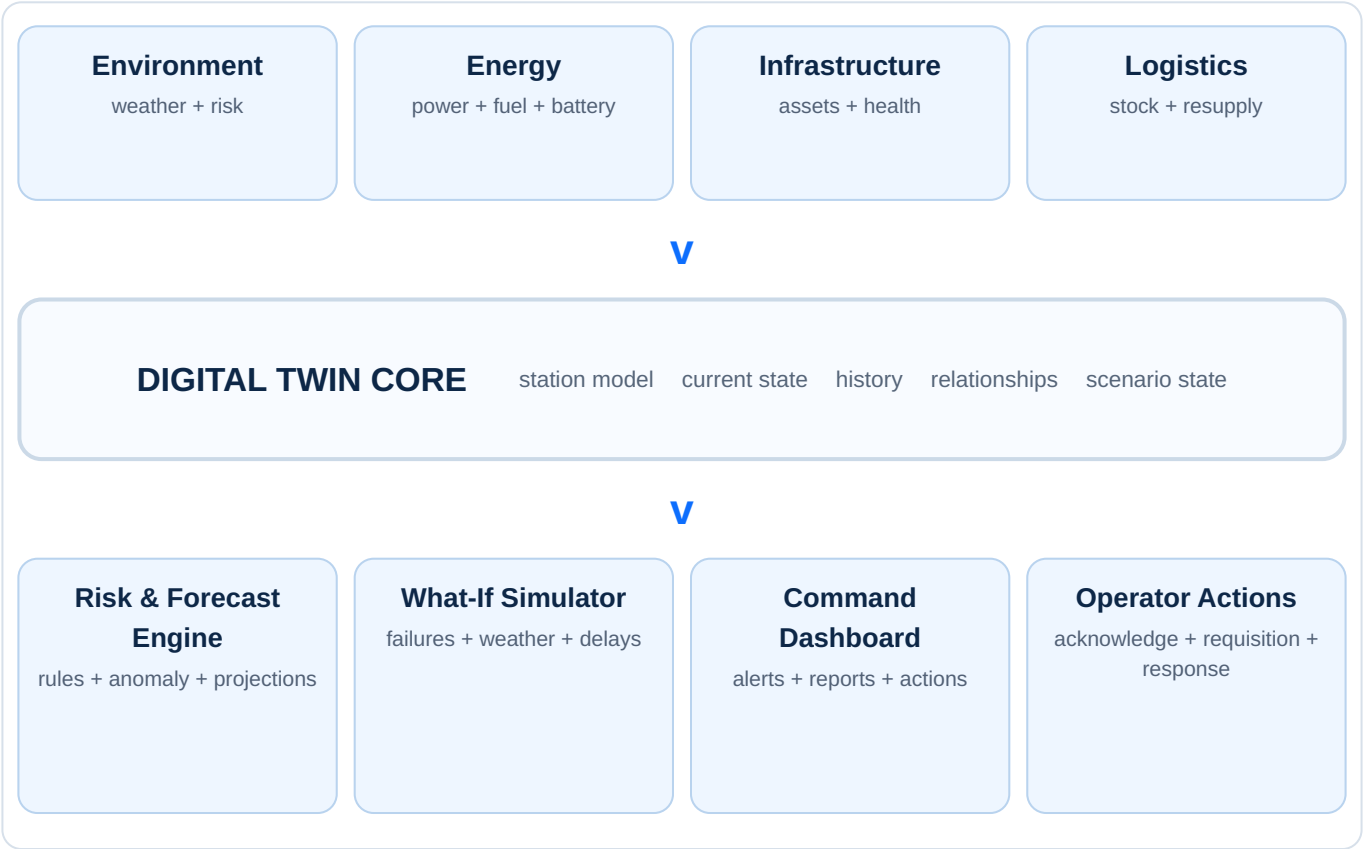
US-01: As an operations user, I can switch between Maitri and Bharati and see a consolidated station health summary.

US-02: I can click a station asset and see its current state, trend and alerts.

US-03: I can see whether power, fuel and critical consumables are likely to last until the next resupply.

US-04: I can run a what-if scenario and see downstream impact before making a decision.

3. MVP Scope and Module Map



MVP modules

Module	Must-have capabilities	Data
1. Command Center	Station toggle, health score, active alerts, critical resource runway, connectivity status.	All domains
2. Digital Twin	2D/3D station model, asset selection, status overlays, twin reset.	Station + assets
3. Environment	Temperature, wind, pressure, visibility, weather risk, trends.	Public/simulated weather
4. Energy	Generator load, generation, battery SOC, fuel, demand, reserve runway.	Simulated telemetry
5. Infrastructure	Asset health, HVAC/water/pump status, maintenance flags, anomalies.	Simulated telemetry
6. Logistics	Inventory, days remaining, requisition workflow, delivery status.	Operational model
7. Risk + Simulator	Rules, alerts, causal chain, what-if scenarios, recommended actions.	Derived state
8. Reports	Daily/incident/energy/logistics reports; CSV/PDF export.	Historical data

4. User Experience and Screen Specification

Route	Screen	Required content
/login	Authentication	Login, role, session expiry, demo credentials banner.
/dashboard	Command Center	Station toggle, twin preview, KPI cards, alert rail, risk summary, critical actions.
/digital-twin	Station Twin	3D/2D view, asset list, asset telemetry, layer filters, alert hotspots.
/environment	Environment	Current weather, trend charts, risk rules, forecast view, station comparison.
/energy	Energy	Generation vs demand, generators, battery, fuel burn, reserve runway.
/infrastructure	Infrastructure	Asset health table, maintenance prediction, HVAC/water systems.
/logistics	Logistics	Inventory ledger, days remaining, requisitions, shipment state.
/simulator	What-If Simulator	Scenario inputs, projection timeline, impact tree, recommended response.
/alerts	Alerts	Filter by severity/domain/station, acknowledgement, timeline.
/reports	Reports	Report type, time window, station, export PDF/CSV.

Command Center layout

+-----+ POLAR COMMAND MAITRI [v] LIVE LAST SYNC 12s AGO ROLE: OPS +-----+		
+-----+ STATION HEALTH POWER FUEL RUNWAY CRITICAL ALERTS CONNECTIVITY +-----+		
 DIGITAL TWIN / STATION MAP [clickable assets] [alert hotspots] +-----+		
+-----+ ENVIRONMENT ENERGY INFRASTRUCTURE -28 C Wind 52 km/h 382 kW demand 18/21 assets healthy Blizzard risk: HIGH 451 kW generation Pump #2 WARNING +-----+		
+-----+ LOGISTICS RISK ENGINE ACTIONS Fuel: 18 days HIGH: 2 [Run Scenario] Food: 31 days WARNING: 4 [Open Alerts] Med: 27 days [Explain risk] [Create Requisition] +-----+		

UX rule: Put critical state above the fold. A user should know station condition, primary risk, resource runway and next action within 10 seconds.

5. Digital Twin Model

Station object hierarchy

```
Station
├── Metadata
│   ├── stationId
│   ├── name
│   ├── location
│   └── connectivityState
├── Buildings
│   ├── MainBuilding
│   ├── SummerCamp
│   └── UtilityModules
├── EnergySystem
│   ├── Generators[]
│   ├── BatteryBank
│   └── LoadGroups[]
├── Infrastructure
│   ├── HVAC
│   ├── WaterPump
│   ├── FuelPump
│   └── CriticalEquipment[]
├── Logistics
│   ├── InventoryItems[]
│   ├── Requisitions[]
│   └── Shipments[]
└── Environment
    ├── Temperature
    ├── Wind
    ├── Pressure
    ├── Visibility
    └── WeatherRisk
```

Asset state contract

```
{
  "assetId": "bharati-gen-01",
  "stationId": "bharati",
  "type": "generator",
  "status": "running",
  "health": 92,
  "metrics": {
    "loadKw": 310,
    "temperatureC": 71.4,
    "fuelRateLph": 18.2
  },
  "alerts": [],
  "updatedAt": "2026-10-02T00:00:00Z"
}
```

State layers

Static

Model geometry, station metadata, asset inventory, thresholds.

Dynamic

Telemetry, weather, battery, generator, stock, connectivity.

Derived

Health score, reserve runway, risk, forecast, recommendations.

Station health score

Use a transparent weighted score for the prototype. Example weights: Environment 20%, Energy 30%, Infrastructure 25%, Logistics 25%.

```
healthScore = 0.20 * environmentScore  
             + 0.30 * energyScore  
             + 0.25 * infrastructureScore  
             + 0.25 * logisticsScore
```

The weights are MVP defaults, not official NCPOR standards. Keep them configurable.

6. Cross-Domain Risk Engine

The Risk Engine is the most important differentiator. It should combine current state, thresholds, trends and dependencies rather than generate isolated alerts.

Risk levels

Level	Meaning	UI
NOMINAL	No action required.	Green state / no interruptive alert.
WARNING	Trend or threshold needs attention.	Amber banner + recommended review.
CRITICAL	Likely operational impact or low reserve.	Red alert + action + acknowledgement.

Core dependency rules

Rule ID	Trigger	Propagation	Example action
R-01	Temperature below configurable threshold	HVAC load + energy demand	Forecast fuel impact.
R-02	Wind/visibility indicates severe weather	Transport/resupply risk	Increase logistics delay estimate.
R-03	Generator unavailable	Generation capacity -> battery/fuel risk	Simulate backup activation.
R-04	Fuel days remaining below buffer	Logistics risk	Raise replenishment priority.
R-05	Critical asset health below threshold	Infrastructure risk	Create maintenance task.
R-06	Resupply delay exceeds buffer	Inventory shortage risk	Escalate requisition.

Example causal chain



Alert payload

```
{
  "alertId": "ALT-00192",
  "stationId": "bharati",
  "severity": "critical",
  "domain": "cross-domain",
  "title": "Fuel reserve may fall below resupply buffer",
  "cause": [
    "External temperature trend increased HVAC demand",
    "Generator 2 unavailable",
    "Fuel runway projected below 14 days"
  ],
  "recommendations": [
    "Activate backup generator",
    "Reduce non-critical load",
    "Review resupply request"
  ],
  "createdAt": "2026-10-02T00:01:20Z"
}
```

7. Simulation and Forecasting Logic

Resource runway

```
fuelRunwayDays = fuelLitres / averageFuelBurnLitresPerDay
inventoryRunwayDays = stockQty / forecastDailyConsumption
batteryRunwayHours = usableKWh / projectedLoadKw
```

Energy model - MVP

```
baseLoadKw = criticalLoadKw + nonCriticalLoadKw
heatingLoadKw = f(outsideTemperature, windSpeed, buildingState)
demandKw = baseLoadKw + heatingLoadKw
netPowerKw = totalGenerationKw - demandKw

if netPowerKw < 0:
    batterySoc = batterySoc - deficitKWh / batteryCapacityKWh
    risk += energyDeficitPenalty
```

Scenario types

Generator failure

Disable one generator and recalculate generation, battery drain, fuel use and critical load coverage.

Extreme cold

Decrease outside temperature and recalculate HVAC load, energy demand and fuel runway.

Resupply delay

Add days to ETA and calculate whether current inventory remains above safety stock.

Asset degradation

Reduce asset health and efficiency to estimate downstream impact.

Simulator interface

SCENARIO: Bharati - Extreme Weather

Temperature adjustment: -10 C
Wind adjustment: +20 km/h
Generator #2 state: OFFLINE
Resupply delay: +5 days

[RUN SIMULATION]

RESULTS

Energy deficit: 9.8%
Battery after 24h: 31%
Fuel runway: 11.7 days
Critical inventory risk: HIGH
Overall operational risk: CRITICAL

RECOMMENDED RESPONSE

1. Start backup generation
2. Reduce non-critical load
3. Escalate fuel replenishment
4. Monitor HVAC and battery reserve

For the hackathon MVP, deterministic simulation is acceptable and often easier to defend than an opaque model. Add ML only where historical data exists or can be responsibly generated for demonstration.

8. Data Strategy

NCPOR publicly exposes polar weather information and a data portal with Maitri and Bharati observations. The portal currently exposes station weather views and links for observations/surface-station data. [2][3]

Three-layer data policy

Source class	Use in MVP	Label
Public NPCOR environmental observations	Use for environment cards/trends when accessible.	"Public observation"
Synthetic energy/infrastructure telemetry	Drive generator, HVAC, battery, water systems and scenarios.	"Simulated telemetry"
Synthetic logistics state	Demonstrate stock, consumption, resupply and requisitions.	"Prototype operational model"

Telemetry generator requirements

- Generate values with continuity, not random jumps.
- Use station-specific baselines and configurable noise.
- Allow injected events such as generator failure, cold snap or resupply delay.
- Persist snapshots for historical charts and reports.
- Expose a "simulation mode" banner in the UI.

Telemetry schema

```
{
  "stationId": "maitri",
  "timestamp": "2026-10-02T00:00:00Z",
  "environment": {
    "temperatureC": -24.6,
    "windKmh": 38,
    "pressureHpa": 965,
    "visibilityKm": 8
  },
  "energy": {
    "generationKw": 420,
    "demandKw": 365,
    "batterySoc": 68,
    "fuelLitres": 8200
  },
  "infrastructure": {
    "hvacHealth": 88,
    "waterPumpHealth": 71,
    "generatorHealth": 92
  }
}
```

Data integrity rule: Never present synthetic operational telemetry as official station telemetry. Make the data source visible in the UI and reports.

9. Backend, APIs and Database

Recommended architecture



API contract

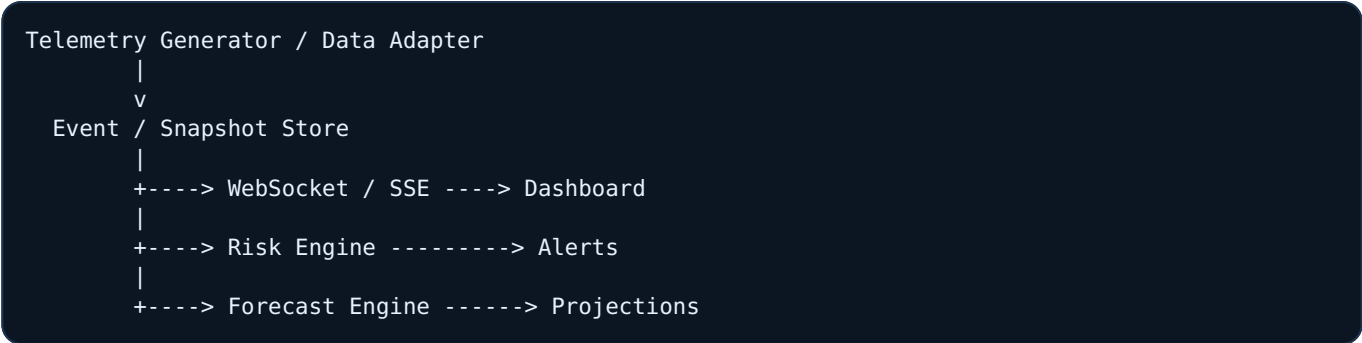
Method	Endpoint	Purpose
GET	/api/stations	List Maitri/Bharati.
GET	/api/stations/:id/state	Current full twin state.
GET	/api/stations/:id/telemetry	Historical telemetry window.
GET	/api/stations/:id/alerts	Active + historical alerts.
POST	/api/simulations	Run what-if scenario.
GET	/api/logistics/:id/inventory	Inventory and runway.
POST	/api/logistics/requisitions	Create requisition.
PATCH	/api/alerts/:id/ack	Acknowledge alert.
GET	/api/reports	Build report metadata.
POST	/api/reports/export	Generate PDF/CSV.

Minimum database tables

Table	Key fields
stations	id, name, latitude, longitude, status, timezone
assets	id, station_id, type, name, status, health, threshold_config
telemetry_snapshots	id, station_id, timestamp, environment_json, energy_json, infra_json
inventory_items	id, station_id, sku, category, quantity, daily_consumption, safety_stock
requisitions	id, station_id, requester_id, priority, status, items_json, eta
alerts	id, station_id, severity, domain, title, cause_json, status, created_at
simulation_runs	id, station_id, scenario, inputs_json, outputs_json, created_at
users	id, name, email, role, password_hash, last_login
audit_logs	id, user_id, action, entity_type, entity_id, timestamp

10. Realtime, Offline and Security Design

Realtime flow



Offline-first behavior

Condition	App behavior
Connected	Receive live updates; show last sync time.
Intermittent	Keep last-known-good state; queue local events where appropriate.
Disconnected	Show offline banner; continue local simulation/alert rules with cached data.
Reconnected	Synchronize buffered events and refresh state.

Security MVP

- JWT or secure session cookies.
- Role-based authorization at API and UI level.
- Audit logs for approvals, inventory changes, alert acknowledgement and report exports.
- No hard-coded secrets in frontend source.
- Environment variables for database/API credentials.

RBAC matrix

Action	Ops	Engineer	Logistics	Leadership
View dashboard	R/W	R/W	R	R
Acknowledge alerts	Yes	Yes	Limited	No
Run simulations	Yes	Yes	Optional	Read
Create requisition	Yes	Optional	Yes	No
Approve requisition	Yes	No	Yes	Optional
Export reports	Yes	Yes	Yes	Yes

11. Frontend Components and Folder Structure

Recommended frontend stack

Framework Next.js or React + TypeScript	UI Tailwind CSS + accessible component primitives	Charts Recharts or ECharts
Twin Three.js / React Three Fiber for optional 3D	State Zustand or Redux Toolkit	Data TanStack Query for server state

Suggested structure

```
app/  
├── (auth)/login/page.tsx  
├── dashboard/page.tsx  
├── digital-twin/page.tsx  
├── environment/page.tsx  
├── energy/page.tsx  
├── infrastructure/page.tsx  
├── logistics/page.tsx  
├── simulator/page.tsx  
├── alerts/page.tsx  
├── reports/page.tsx  
components/  
├── station/StationSelector.tsx  
├── twin/StationTwin.tsx  
├── twin/AssetInspector.tsx  
├── cards/HealthCard.tsx  
├── charts/TelemetryChart.tsx  
├── alerts/AlertRail.tsx  
├── simulator/ScenarioForm.tsx  
├── simulator/ImpactGraph.tsx  
lib/  
├── api.ts  
├── rules.ts  
├── calculations.ts  
├── permissions.ts  
├── simulator.ts  
mocks/  
├── stationData.ts  
├── telemetryGenerator.ts  
public/  
├── twin/maitri.glb  
├── twin/bharati.glb
```

UI quality rules

- Desktop-first command center, responsive for tablet.
- Dark navy + ice-blue visual language; use color only as a secondary status cue.
- Charts must have clear units, timestamps and source labels.
- All alerts need a human-readable reason.
- Use loading, empty and offline states on every data-heavy screen.
- Do not overload the 3D view; keep a 2D fallback for low-end devices.

12. Five-Minute Demo Flow

Time	Action	What judge should see
0:00-0:30	Open Command Center.	Two stations, health, environment, energy, logistics, alerts.
0:30-1:15	Open Bharati twin and inspect Generator #2.	Spatial asset + live telemetry + health trend.
1:15-2:00	Trigger extreme cold event.	HVAC demand rises and energy state changes automatically.
2:00-2:45	Show risk engine.	Fuel runway drops; logistics warning appears with cause chain.
2:45-3:45	Run "Generator failure + resupply delay" scenario.	System projects battery, fuel, inventory and risk changes.
3:45-4:30	Create/review requisition.	Role-based logistics workflow.
4:30-5:00	Export incident report.	Professional PDF/CSV output + audit trail.

Demo narrative

Opening: "A remote operator in India cannot physically inspect the station. Polar Command gives that operator a connected digital representation of the station's condition and the consequences of possible failures."

Middle: "This cold-weather event is not just a weather alert. It changes heating demand, power balance, fuel consumption and resupply risk. The twin propagates the state change."

Close: "Before an operator commits to an action, the simulator tests the scenario and shows the safest operational response."

13. Acceptance Criteria / Definition of Done

ID	Acceptance test
AC-01	User can switch between Maitri and Bharati without page reload.
AC-02	Dashboard shows current state across all four domains.
AC-03	Telemetry updates automatically and shows last-sync timestamp.
AC-04	At least one environmental event changes energy and logistics outputs.
AC-05	At least one asset failure creates a cross-domain alert.
AC-06	Risk engine classifies alerts as Nominal/Warning/Critical.
AC-07	Every alert shows causes and one or more recommendations.
AC-08	User can run at least three scenarios and compare baseline vs scenario.
AC-09	Fuel and inventory runway calculations are deterministic and testable.
AC-10	Logistics requisition has create/review/deliver states.
AC-11	RBAC prevents unauthorized actions.
AC-12	Offline state is visible and last-known state remains usable.
AC-13	Reports export to PDF/CSV with source labels.
AC-14	No console errors during the 5-minute demo path.
AC-15	Application can run using seed data with no external API dependencies.

MVP priority

Build first

Command Center + telemetry generator + risk engine + energy/fuel model + logistics runway + simulator.

Build second

3D twin refinement + public weather adapter + advanced forecasting + polished reports.

Out of scope for MVP

- Real commanding/control of physical station equipment.
- Unverified integration with private NCPOR systems.
- High-fidelity structural engineering simulation.
- Production-grade satellite communications stack.
- Autonomous safety decisions without human approval.

14. Build Plan for Antigravity

Implementation sequence

Phase	Deliverable	Checkpoint
1	Scaffold app, theme, routing, auth, seed data.	All routes load.
2	Station state + telemetry generator + realtime stream.	Dashboard numbers update continuously.
3	Energy + infrastructure calculations.	Generator/fuel/battery dependencies work.
4	Logistics + inventory runway + requisitions.	Stock and resupply workflow works.
5	Risk engine + causal explanations.	Cross-domain alert triggered by injected event.
6	Simulator.	Scenario changes propagate through the model.
7	Digital twin UI/3D.	Assets clickable and tied to live state.
8	Reports + audit + offline polish.	Demo path passes acceptance tests.

Antigravity working rules

- Implement small vertical slices and run the app after each phase.
- Keep simulation logic separate from UI components.
- Use typed interfaces for telemetry, assets, alerts, inventory and scenarios.
- Prefer deterministic seeded simulation for reproducible demos.
- Write unit tests for formulas and rules before polishing charts.
- Keep all thresholds and weights configurable from one configuration module.

Recommended branch/commit milestones

```
feat/001-scaffold
feat/002-telemetry-engine
feat/003-energy-infrastructure
feat/004-logistics
feat/005-risk-engine
feat/006-simulator
feat/007-digital-twin
feat/008-reports-and-hardening
```

15. Copy-Paste Master Prompt for Antigravity

Paste the following as the master build brief. Then ask the coding agent to execute the phases one by one, validating the app after each phase.

You are the lead full-stack engineer building an SIH 2026 prototype for problem statement SIH26060.

PRODUCT NAME: POLAR COMMAND

PRODUCT GOAL:

Build a web-based Digital Twin platform for efficient remote management of India's Antarctic research stations Maitri and Bharati. Integrate four domains into one connected operational model:

- 1) environment
- 2) energy
- 3) infrastructure
- 4) logistics

CORE PRODUCT LOOP:

OBSERVE -> UNDERSTAND -> PREDICT -> SIMULATE -> RESPOND

IMPORTANT PRODUCT INTERPRETATION:

This is NOT a generic dashboard and NOT a 3D-only demo. The Digital Twin must connect the domains. Example: colder weather increases heating demand, which changes power demand, fuel consumption, reserve runway and logistics risk.

MVP MODULES:

1. Command Center
2. Digital Twin
3. Environment
4. Energy
5. Infrastructure
6. Logistics
7. Risk Engine
8. What-If Simulator
9. Alerts
10. Reports

PRIMARY USERS:

- NCPOR Operations
- Station Engineer
- Logistics Officer
- Leadership

Implement RBAC for these roles.

FRONTEND:

Use Next.js/React + TypeScript + Tailwind CSS.

Use Recharts/ECharts for telemetry.

Use React Three Fiber/Three.js for the optional 3D twin, but always provide a 2D fallback.

Use TanStack Query for server state and a lightweight client store for UI state.

BACKEND:

Use Node.js/TypeScript or FastAPI.

Expose REST APIs and WebSocket/SSE for realtime telemetry.

Use PostgreSQL. Keep persistence interfaces clean enough to swap the database if needed.

CORE ROUTES:

/login
/dashboard
/digital-twin
/environment
/energy
/infrastructure

/logistics
/simulator
/alerts
/reports

CORE API:

GET /api/stations
GET /api/stations/:id/state
GET /api/stations/:id/telemetry
GET /api/stations/:id/alerts
POST /api/simulations
GET /api/logistics/:id/inventory
POST /api/logistics/requisitions
PATCH /api/alerts/:id/ack
GET /api/reports
POST /api/reports/export

DATA MODEL:

Station -> Buildings -> EnergySystem -> Infrastructure -> Logistics -> Environment
Dynamic state must be separate from static metadata and derived state.

TELEMETRY:

Create a deterministic telemetry simulator for Maitri and Bharati.
Generate coherent temperature, wind, pressure, visibility, generator load, generation, demand, battery SOC, fuel volume, HVAC health, water pump health and inventory consumption.
Do not use arbitrary random jumps. Use bounded noise and gradual state changes.

Allow injected events:

- extreme cold
- high wind / blizzard risk
- generator failure
- asset degradation
- resupply delay

DATA HONESTY:

Mark synthetic operational data as SIMULATED TELEMETRY or PROTOTYPE OPERATIONAL MODEL.
Do not imply private station telemetry is public.
Where public NCPOR weather observations are accessible, create an adapter interface so they can be used for environment data.
The app must also run fully from local seed/simulated data without any external API.

RISK ENGINE:

Use transparent rules first. Support three severities:

NOMINAL, WARNING, CRITICAL.

Every alert must include:

- title
- severity
- domain
- causes
- affected assets/domains
- recommended actions
- timestamp

Required cross-domain rules:

- temperature -> HVAC load -> energy demand -> fuel burn
- generator failure -> generation deficit -> battery/fuel risk
- severe weather -> resupply/transport risk
- inventory below safety stock -> logistics risk
- critical asset health -> maintenance risk

STATION HEALTH:

Compute configurable health score from Environment, Energy, Infrastructure and Logistics.
Default weights may be 20%, 30%, 25%, 25% but keep them configurable.

SIMULATOR:

Create a What-If screen where the user can modify:

- temperature
- wind

- generator availability
- resupply delay
- asset health

Then calculate baseline vs scenario for:

- power deficit
- battery SOC
- fuel runway
- inventory runway
- risk level
- recommended actions

ENERGY CALCULATIONS:

$\text{fuelRunwayDays} = \text{fuelLitres} / \text{averageFuelBurnLitresPerDay}$

$\text{inventoryRunwayDays} = \text{quantity} / \text{forecastDailyConsumption}$

$\text{batteryRunwayHours} = \text{usableKWh} / \text{projectedLoadKw}$

$\text{netPowerKw} = \text{totalGenerationKw} - \text{demandKw}$

If $\text{netPowerKw} < 0$, decrease battery SOC according to deficit.

LOGISTICS:

Track fuel, food, medical supplies, water-treatment consumables and spare parts.

Show quantity, daily consumption, safety stock, days remaining, next resupply ETA and risk.

Support requisition states:

DRAFT -> SUBMITTED -> APPROVED -> IN_TRANSIT -> DELIVERED

OFFLINE / EDGE UX:

Show last sync time.

When disconnected, retain last-known-good state.

Keep the simulator and local risk rules usable with cached data.

On reconnection, synchronize state.

SECURITY:

Use secure sessions/JWT, RBAC and audit logs.

Never put secrets in frontend source.

Use environment variables.

Log meaningful actions such as requisition approval, inventory changes, alert acknowledgement and report exports.

UI DESIGN:

Use a professional polar operations visual language: dark navy, ice-blue, white, restrained status colors.

Command Center must surface critical state above the fold.

Charts require units, timestamps and source labels.

All data-heavy pages need loading, empty, offline and error states.

COMMAND CENTER MUST SHOW:

- station selector
- station health
- connectivity / last sync
- digital twin preview
- environment summary
- energy summary
- infrastructure summary
- logistics summary
- active alerts
- next recommended action

DEMO SCENARIO:

1. Start with Bharati nominal.
2. Inject extreme cold.
3. Show HVAC demand increasing.
4. Show generator/power balance changing.
5. Show fuel runway shrinking.
6. Delay resupply.
7. Show cross-domain risk becoming CRITICAL.
8. Run a what-if scenario with Generator #2 OFFLINE.
9. Show projected battery/fuel/inventory impact.
10. Show recommended response.

11. Export an incident report.

ACCEPTANCE CRITERIA:

- switching station works without reload
- realtime telemetry updates
- cross-domain propagation works
- three severity levels work
- alerts have explanations and actions
- simulator changes outputs
- logistics workflow works
- RBAC works
- offline state is visible
- reports export
- no hard dependency on external APIs
- no console errors in the demo path

ENGINEERING REQUIREMENTS:

- TypeScript strict mode where possible
- reusable components
- schema validation for API payloads
- tests for formulas and risk rules
- deterministic seeded demo mode
- clear separation of UI, domain logic, data access and simulation engine
- do not hide important calculations inside components
- add comments only where they clarify non-obvious domain logic

BUILD ORDER:

Phase 1: scaffold, routing, theme, auth, seed data

Phase 2: telemetry engine + realtime

Phase 3: energy + infrastructure

Phase 4: logistics

Phase 5: risk engine

Phase 6: simulator

Phase 7: digital twin

Phase 8: reports, offline behavior, audit and polish

WORK MODE:

Implement one phase at a time.

After every phase, run lint, type-check, tests and the app.

Fix errors before moving on.

Do not replace working domain logic with mock UI merely to make a page look finished.

At the end of every phase, provide:

- 1) files changed
- 2) features completed
- 3) tests/checks run
- 4) remaining risks
- 5) next phase recommendation

Start with Phase 1 and build the project skeleton plus seed data and typed domain models. Do not implement all phases in one uncontrolled change.

16. Recommended Final MVP Deliverable

Core demo

Command Center + Digital Twin + cross-domain Risk Engine + What-If Simulator.

Operational proof

Energy runway + inventory runway + requisition workflow + incident report.

Final checklist before SIH demonstration

- ☐ Maitri and Bharati switch correctly
- ☐ Twin assets map to real data objects
- ☐ Telemetry updates visibly
- ☐ Source label is visible
- ☐ Environment changes affect energy
- ☐ Energy changes affect fuel runway
- ☐ Fuel runway affects logistics risk
- ☐ At least one generator failure scenario works
- ☐ At least one severe-weather scenario works
- ☐ Alerts explain their causes
- ☐ Recommendations are deterministic
- ☐ Simulator compares baseline/scenario
- ☐ Requisition workflow works
- ☐ RBAC blocks unauthorized actions
- ☐ Offline banner/last sync works
- ☐ Report export works
- ☐ Seed data can reset the entire demo
- ☐ No external API is required to demo
- ☐ No broken routes
- ☐ No console errors

Positioning line for the final presentation

"Polar Command turns four isolated operational domains into one living station model - so remote teams can see the present, understand the risk, test the future, and act."

References

- SIH26060 public problem-statement listing: Digital Platform for efficient remote management of Indian Antarctic Research Stations; MoES / NCPOR. [1]
- NCPOR Weather at Indian Polar Stations data portal, including Maitri and Bharati. [2]
- NCPOR Polar Data Centre meteorological observations page, including Maitri and Bharati datasets. [3]

Note: Architecture, thresholds, health-score weights and synthetic telemetry definitions in this document are prototype design choices, not official NCPOR operating standards.